YojanaBench

The First Benchmark for India Welfare-Scheme Document Reasoning + a Multi-Agent Audit Pipeline

FULL DETAILED PROJECT DESCRIPTION · FINAL YEAR ENGINEERING PROJECT

Team Members: [Member 1], [Member 2], [Member 3] | **Guide:** [Guide Name, Department]
College: [College Name] | **Duration:** 4–5 months (19 weeks) | **Domain:** Agentic AI / NLP / E-Governance

Abstract: India runs ~950+ central and state welfare schemes governed by dense PDF guideline documents, yet no structured benchmark exists to test whether AI agents can reason correctly over these documents. YojanaBench addresses this gap with two deliverables: (1) a human-verified benchmark of real scheme guideline PDFs with QA pairs, counterfactual/edge-case scenarios, and verdict references, and (2) a reproducible multi-agent audit pipeline (Corpus, Intake, RAG, Entitlement, Verification, Orchestrator, Evaluation, and Appeal agents) built on LangGraph + Ollama with zero API cost, evaluated using Exact Match, F1, verdict accuracy, entitlement error, and ablation studies.

Table of Contents

1.	Introduction & Background
2.	Problem Statement
3.	Research Gap & Novelty (Verified Uniqueness)
4.	Objectives
5.	System Architecture
6.	Detailed Agent Descriptions
7.	Benchmark Dataset Design (Deliverable 1)
8.	Evaluation Protocol & Metrics (Deliverable 2)
9.	Technology Stack
10.	Work Plan & Timeline (19 Weeks)
11.	Expected Outcomes & Deliverables
12.	Novelty, Paper Potential & Comparisons
13.	Risk Analysis & Mitigation
14.	Feasibility & FYP Suitability
15.	Possible Extensions & Future Work

1. Introduction & Background

1.1 Welfare-Scheme Landscape in India

The Government of India and state governments together run approximately **950+ welfare schemes** covering food security (ration/PDS), old-age/widow/disability pensions, scholarships, employment guarantee (MGNREGA), housing (PMAY), farmer aid (PM-KISAN), health insurance (Ayushman Bharat), and beneficiary assistance programs. Each scheme is defined and operated through official **guideline documents** — typically long PDFs issued by the relevant ministry or department.

1.2 Role of Guideline Documents

These guideline PDFs are the legal and administrative source of truth for:

- **Eligibility criteria** — age limits, income ceilings, category (SC/ST/OBC/General), residency, family income, landholding.
- **Entitlement formulas** — exact benefit amounts: pension tiers, ration quantum per member, scholarship fee waivers, wage rates.
- **Document requirements** — Aadhaar, income certificate, caste certificate, Ration card, bank passbook, domicile proof.
- **Application & appeal procedure** — filing steps, timelines, revision/appeal hierarchy (Block → District → State Commission).

1.3 Real-World Failure Modes

Failure	Consequence
Eligibility misreading	Wrongly approved (leakage) or wrongly rejected (exclusion) applications
Wrong entitlement calculation	Incorrect benefit amounts paid to crores of beneficiaries
Mis-routing of applications	Cases sent to wrong department/channel; months of delay
No quality test for AI systems	Any deployed AI tool is untested against a ground standard

1.4 Why Now

LLM-based assistants and RAG systems are being proposed for citizen-facing governance. Without a benchmark, there is **no way to measure** whether such a system reasons correctly over scheme rules. YojanaBench provides that measuring instrument — for our own pipeline and for any future research in this space.

2. Problem Statement

India runs ~950+ central/state welfare schemes (ration, pensions, scholarships, employment, beneficiary aid) governed by dense PDF guideline documents. Every year, **eligibility errors, wrong entitlement calculation, and mis-routing of applications cause huge delays**. Yet there is **no structured benchmark** to test whether AI agents can reason correctly over these real documents — existing benchmarks are English-general (GAIA) or Chinese-law (LawBench). Consequently:

1. Researchers cannot compare document-reasoning methods on Indian welfare documents.
2. Deployed AI tools cannot be validated for eligibility and entitlement correctness.
3. Citizens bear the cost of errors in the form of rejected claims, wrong payments, and delayed appeals.

Project goal: build the missing benchmark (dataset + protocol) *and* a multi-agent audit pipeline that performs the full application-audit workflow — intake, retrieval, entitlement computation, verification, verdict, evaluation, and appeal routing — and demonstrate its correctness against the benchmark.

3. Research Gap & Novelty (Verified Uniqueness)

3.1 Existing Benchmarks and Their Limits

Benchmark	Domain	Why Not Sufficient
GAIA	General AI assistant tasks (English)	No domain documents; no eligibility/entitlement reasoning; not India-specific
LawBench	Chinese legal reasoning	Chinese law; different legal structure; no welfare schemes
SQuAD / doc-QA benchmarks	Reading comprehension	Extractive only; no verdicts, entitlement math, or counterfactuals
Indian NLP work (PDS/nutrition)	Government MIS portals	Operates on portal data, not guideline-PDF reasoning; no QA benchmark
YojanaBench (ours)	**India welfare-scheme guideline PDFs**	**First benchmark: QA + counterfactuals + verdict ground truth + evaluation protocol**

3.2 Verification Statement

Literature search (2025–26) across NLP/agent venues found **no India-specific welfare-scheme document-compliance benchmark**. This makes the dataset itself a contribution that nobody can dismiss as "already existing."

3.3 Three Novel Contributions

#	Contribution	Type
(a)	YojanaBench dataset — real scheme PDFs, human-verified QA, counterfactuals, verdict references	Data contribution
(b)	Evaluation protocol/metrics — EM, F1, verdict accuracy, entitlement error, calibration, ablation design	Methodological
(c)	Reproducible open-source multi-agent audit pipeline (Ollama + LangGraph, zero API cost)	Systems

4. Objectives

1. Shortlist and collect **10–15 high-volume scheme guideline PDFs** (central + state; PDS, pensions, scholarships, employment, beneficiary aid).
2. Build a structured **scheme schema** (eligibility clauses, entitlement formulas, document checklist, appeal hierarchy) per scheme.
3. Create **human-verified QA pairs** across 4 categories: eligibility, entitlement, documents, appeal steps.
4. Design **counterfactual and edge-case scenarios** (boundary incomes, missing documents, overlapping schemes) with ground-truth verdicts.
5. Implement the **multi-agent audit pipeline** (Corpus, Intake, RAG, Entitlement, Verification, Orchestrator, Evaluation, Appeal agents).
6. Define and run the **evaluation protocol** (EM, F1, verdict accuracy, entitlement error, confidence calibration).
7. Run **baselines and ablations** (single-LLM vs multi-agent; agent-removal studies).
8. Build a **dashboard + demo scenarios** showing approve / reject / appeal flows.
9. Release the benchmark and pipeline with documentation, licenses, and statistics.

5. System Architecture

5.1 End-to-End Workflow

```
Scheme PDFs (public)
    -> [Corpus Agent]  chunking + schema extraction  ->  vector store

A case application
    -> [Intake Agent]      extract fields + identify needed documents
    -> [RAG Agent]         fetch eligibility / entitlement clauses
    -> [Entitlement Agent] compute benefit (amount / ration / fee)
    -> [Verification Agent] missing docs / ineligible flags (cross-check)
    -> [Orchestrator]      verdict + explanation + confidence
                              (human-approval gate for low-confidence cases)
    -> [Evaluation Agent]  score vs YojanaBench ground truth
                           (EM / F1 / verdict accuracy / entitlement error)
    -> [Appeal Agent]      route rejection/dispute to correct appeal channel
```

5.2 Two-Phase Design

Phase	Trigger	Agents Involved
Offline corpus phase	One-time + when a new scheme is added	Corpus Agent → vector store
Online case phase	Each incoming application / benchmark question	Intake → RAG → Entitlement → Verification → Orchestrator → (Evaluation / Appeal)

5.3 Data Structures Passed Between Agents

Object	Fields
SchemeSchema	scheme_id, name, state/central, eligibility_clauses[], entitlement_rules[], required_documents[], appeal_hierarchy[], source_pdf_pages
CaseRecord	case_id, applicant fields (age, income, category, region, family size, landholding), submitted_documents[], scheme_id
Verdict	decision (eligible / ineligible / needs_info), explanation, evidence_clause_ids[], computed_entitlement, confidence (0–1), missing_documents[]
ScoreReport	em, f1, verdict_accuracy, entitlement_error, calibration_delta, agent_trace

6. Detailed Agent Descriptions

6.1 Corpus Agent (Ingestion)

- **Input:** scheme guideline PDF files (public government documents).
- **Functions:** PDF text extraction (PyMuPDF/pdfplumber) → section-aware chunking (by clause/section headers, not fixed tokens) → schema extraction using LLM (eligibility clause, entitlement rule, document item, appeal step) → embedding into vector store with metadata (scheme_id, section type, page).
- **Output:** vector store + structured SchemeSchema JSON per scheme.
- **Why special chunking:** scheme rules are clause-structured; naive chunking splits a rule from its condition — section-aware chunking preserves rule integrity.

6.2 Case-Intake Agent

- **Input:** a case application (form fields or unstructured text) + attached document list.
- **Functions:** field extraction (age, income, category, region, family size) with validation; detect target scheme; identify the document checklist the case *should* have; flag missing submissions early.
- **Output:** CaseRecord (structured fields + needed_documents vs submitted_documents diff).
- **Edge handling:** ambiguous values (e.g., income in "lakhs" vs rupees) normalised; unknown fields marked, not guessed.

6.3 RAG Retrieval Agent

- **Input:** CaseRecord + scheme_id.
- **Functions:** hybrid retrieval (dense vector + keyword/BM25) over scheme chunks; re-ranking to top-k clauses; returns eligibility clauses relevant to the applicant's profile and entitlement rules for the scheme.
- **Output:** evidence set: clause IDs + text + source pages.
- **Why it matters:** verdicts must be grounded in retrieved clauses (explainability + citation), not model memory.

6.4 Entitlement-Calculator Agent

- **Input:** evidence clauses + applicant fields.
- **Functions:** convert retrieved benefit rules into executable computations (e.g., pension = tier(amount by age+category); ration = quantum per member × family size; scholarship = fee − waiver%). Applies conditions (income ceiling, BPL/APL status) and computes the exact benefit.
- **Output:** computed_entitlement (amount/quantity) + the formula + clause IDs used.
- **Design rule:** arithmetic is done by deterministic code where possible; LLM only maps rule → formula (reduces hallucinated math).

6.5 Verification Agent

- **Input:** CaseRecord + retrieved clauses + computed entitlement.
- **Functions:** (a) document completeness check against required checklist; (b) eligibility condition check (age/income/category/residency) per clause; (c) cross-check: re-derive eligibility

independently from clauses and compare with Entitlement Agent's result (multi-agent disagreement forces “needs_info”).

- **Output:** missing_documents[], ineligible_flags[], cross-check status (agree/disagree).

6.6 Orchestrator (+ Human-Approval Gate)

- **Framework:** LangGraph state machine controlling agent order, retries, and conditional branches.
- **Functions:** merges agent outputs into a final **Verdict** (decision + explanation + evidence + confidence); enforces a **human-approval gate** when confidence < threshold or when Verification reports disagreement.
- **Output:** Verdict object; full agent trace stored for auditability.

6.7 Evaluation Agent

- **Input:** Verdict + YojanaBench ground truth.
- **Functions:** computes EM, token-F1, verdict accuracy, entitlement error (absolute/relative), confidence calibration; aggregates per scheme, per QA category; produces results tables/plots and ablation comparisons.
- **Output:** ScoreReport (per item + aggregate).

6.8 Dispute / Appeal-Routing Agent

- **Input:** rejection Verdict + scheme's appeal hierarchy.
- **Functions:** determine correct appeal channel (e.g., Block Development Officer → District Magistrate → State Social Welfare Commission), generate draft appeal text citing the rejected clause, note deadlines.
- **Output:** appeal routing instruction + draft appeal letter for human review.

7. Benchmark Dataset Design (Deliverable 1)

7.1 Corpus Composition

Attribute	Plan
Schemes	10–15 high-volume schemes (mix of central + state; PDS/ration, pensions, scholarships, employment, beneficiary aid)
Source documents	Real, publicly available guideline PDFs (govt websites); licenses noted
Schema per scheme	eligibility clauses, entitlement rules, document checklist, appeal hierarchy
Statistics to report	#schemes, #PDFs, #pages, #chunks, #QA pairs, category distribution, inter-annotator agreement

7.2 QA Pair Categories (with examples)

Category	Question Type	Example
Eligibility	Is X eligible under condition Y?	"A 58-year-old widow with annual income ₹95,000 — eligible for X pension scheme?"
Entitlement	Compute the exact benefit	"Family of 5, APL card — how many kg rice per month at what price?"
Documents	List required documents	"Which certificates must accompany a scholarship application for Category-Z students?"
Appeal	Procedure/steps/deadlines	"Application rejected at Block level — what is the next appeal step and time limit?"

7.3 Counterfactual & Edge Cases

- **Boundary values:** income exactly at the ceiling (₹1.00 lakh vs ₹1,00,001); age exactly at cut-off (59 vs 60).
- **Missing documents:** eligible on criteria but lacks one mandatory certificate → correct verdict is "needs_info," not "eligible."
- **Category swaps:** same profile under SC vs General quota changes entitlement tier.
- **Overlapping schemes:** applicant eligible for two benefits; benchmark expects correct identification.
- **Distractor schemes:** question about scheme A answered using scheme B's rules → tests retrieval precision.
- **Negation/trick phrasing:** "not exceeding," "at least," double negatives in clauses.

7.4 Ground Truth Format

```
{
  "id": "YB-PDS-0042",
  "scheme_id": "PDS-APL-2023",
  "category": "entitlement" | "eligibility" | "documents" | "appeal",
  "question": "...",
  "reference_answer": "...",
  "verdict": "eligible" | "ineligible" | "needs_info",
  "entitlement_value": 5.0,
  "entitlement_unit": "kg/month",
  "evidence_clause_ids": ["PDS-APL-C12", "PDS-APL-C14"],
  "counterfactual_of": null,
  "edge_case": "boundary_income" | null,
  "annotator_agreement": true
}
```

7.5 QA Production & Verification Process

1. **Authoring:** team members draft QA pairs directly from scheme PDFs, citing page/clause.
2. **Independent check:** second member verifies answer against the source clause.
3. **Adjudication:** disagreements resolved jointly with the guide; only agreed items enter the benchmark.
4. **Agreement metric:** report % inter-annotator agreement as a dataset-quality statistic.
5. **Counterfactual generation:** each base QA item gets ≥1 perturbed variant with updated ground truth.

8. Evaluation Protocol & Metrics (Deliverable 2)

Metric	Applies To	Definition
Exact Match (EM)	QA answers	1 if predicted answer matches reference after normalisation, else 0
Token-level F1	QA answers	Harmonic mean of token precision/recall vs reference
Verdict Accuracy	Eligibility decisions	% of cases where (eligible/ineligible/needs_info) matches ground truth — **primary metric**
Entitlement Error	Computed benefits	|predicted − correct| (absolute) and relative % error on amounts/quantities
Document F1	Checklist outputs	F1 over the set of required documents identified
Confidence Calibration	All verdicts	Gap between stated confidence and observed accuracy (ECE-style)

8.1 Experimental Design

Experiment	Setup	Purpose
Baseline: single LLM	One prompt, full document context, no agents	Show value of the multi-agent pipeline
Baseline: naive RAG	Standard fixed-chunk RAG + answer	Show value of schema/specialised agents
Full pipeline	All 8 agents	Main result
Ablation: no Verification	Remove cross-check agent	Measure contribution of verification step
Ablation: no Entitlement (LLM-only math)	Let LLM compute directly	Measure hallucinated-math reduction
Ablation: no counterfactuals in analysis	Score only base QA	Show models fail disproportionately on edge cases
Per-category breakdown	Scores by eligibility/entitlement/documents/appeal	Identify weakest reasoning type

8.2 Expected Result Tables/Plots

- Bar chart: EM / F1 / Verdict Accuracy — baseline vs naive RAG vs full pipeline.
- Ablation drop chart (percentage-point loss per removed agent).
- Entitlement error distribution (histogram).
- Per-scheme and per-category heat map.
- Calibration curve (stated confidence vs accuracy).

9. Technology Stack

Layer	Choice	Reason
Agent orchestration	**LangGraph**	Stateful graphs, conditional branches, human-in-the-loop interrupts
LLM	**Ollama** (Llama 3 / Qwen, 7–8B)	Local, free, reproducible — **zero API cost**
PDF processing	PyMuPDF / pdfplumber	Reliable text + table extraction from govt PDFs
Vector store	Chroma / FAISS	Lightweight local embedding store
Retrieval	Dense + BM25 hybrid, re-ranker	Clause names & section numbers favour keyword search; semantics favour dense
Entitlement math	Python deterministic functions	Prevents LLM arithmetic hallucination
Backend	Python + FastAPI	Serves pipeline to dashboard
Frontend	React dashboard	Case entry, verdict display, metrics view, demo scenarios
Storage	PostgreSQL / JSON files	Ground truth, agent traces, score reports
Versioning	Git + DVC-style data manifests	Reproducibility for defence

10. Work Plan & Timeline (19 Weeks)

Phase	Weeks	Activities	Deliverable
1. Scheme shortlist & doc collection	1–3	Choose 10–15 schemes; download official PDFs; extract schema; build corpus pipeline	Curated PDF corpus + SchemeSchema JSONs
2. QA + human verification + edge cases	4–8	Draft QA across 4 categories; independent verification; counterfactual generation; agreement stats	Benchmark v1 with ground truth
3. Agent development	6–11	Corpus, Intake, RAG, Entitlement, Verification, Orchestrator (overlaps QA phase)	Working end-to-end pipeline
4. Evaluation & metrics	10–14	Evaluation Agent, baselines, ablations, plots	Results tables + ablation charts
5. Dashboard + demo scenarios	14–16	React UI, 3–5 showcase cases (approve/reject/appeal)	Live demo
6. Report & defence	17–19	Robustness testing, documentation, final report, PPT, rehearsal	Final report + defence

Team allocation (3 students): Member A — dataset/corpus + QA coordination; Member B — agents (Intake/RAG/Entitlement); Member C — Verification/Orchestrator + evaluation + dashboard. QA drafting shared by all three.

11. Expected Outcomes & Deliverables

1. **YojanaBench dataset:** 10–15 scheme corpus, human-verified QA pairs, counterfactual/edge cases, verdict ground truth, agreement statistics, documented licenses.
2. **Multi-agent audit pipeline:** 8 agents working end-to-end with agent traces and confidence-scored verdicts.
3. **Quantitative results:** baseline vs pipeline comparison, ablation tables, per-category breakdowns, calibration analysis.
4. **Dashboard + demo:** case entry → verdict → appeal routing walkthrough.
5. **Documentation:** report, PPT, reproducibility instructions (run with Ollama only).

12. Novelty, Paper Potential & Comparisons

Aspect	YojanaBench	GAIA	LawBench
Domain	India welfare-scheme PDFs	General assistant tasks	Chinese law
Verdict ground truth	**Yes (eligible/ineligible/needs_info)**	No	Partially
Numerical entitlement tasks	**Yes**	No	No
Counterfactual/edge cases	**Yes (built-in)**	Limited	Limited
Multi-agent evaluation pipeline	**Included**	No	No
Zero-cost reproducibility	**Yes (Ollama)**	Varies	Varies

Paper-ability: results are strongly examinable — dataset statistics, metric tables, ablation studies, and qualitative error analysis form a complete empirical paper. Suitable for NLP/agent workshops and student research venues.

13. Risk Analysis & Mitigation

Risk	Impact	Mitigation
Document ambiguity / contradictory clauses	Wrong ground truth	Scope to 10–15 high-volume, well-documented schemes; tightened QA with independent verification and adjudication
Licensing of government PDFs	Release blocked	All sources public govt documents; licenses explicitly noted in dataset card
LLM hallucination on arithmetic	Wrong entitlements	Entitlement Agent uses deterministic code; LLM only maps rule→formula; Verification cross-check
Scope creep (950+ schemes)	Timeline overrun	Strict MVP boundary of 10–15 schemes; Corpus agent designed to scale later
QA phase labour-intensive	Delay	Parallelised across 3 members; agent development overlaps from week 6
Local LLM (7–8B) weak reasoning	Low scores	Acceptable — benchmark is meant to expose limits; report per-model comparison; hybrid clause-grounding
Retrieval misses the right clause	Incorrect verdicts	Hybrid retrieval + re-ranking; evidence-clause recall reported as a diagnostic metric

14. Feasibility & FYP Suitability

- **Zero API cost: Ollama + LangGraph + open-source vector store — fully reproducible on a normal laptop.**
- **Data accessibility: all scheme PDFs are public; no private/government API access needed.**
- **Strong examinability: dataset statistics, metric tables, ablation charts, live demo — ideal defence material.**
- **Team fit: 3 students / 19 weeks with clear parallel workstreams (dataset / agents / evaluation-dashboard).**
- **Clear two-deliverable structure: data contribution + systems contribution; both independently gradeable.**

15. Possible Extensions & Future Work

- **Multilingual QA:** questions/answers in Hindi + regional languages (tests cross-lingual document reasoning).
- **Scale the corpus:** add schemes incrementally; measure benchmark growth effects.
- **Vision modality:** handle scanned PDFs and application-form images (VLM + OCR).
- **Human study:** compare agent verdicts with actual welfare-officer decisions.
- **Model comparison:** benchmark across larger open models and closed APIs (optional, reported separately).

- **Deployment pathway:** integrate with citizen-facing portals as a decision-support (not decision-making) tool with the human-approval gate.
